

NYC Taxi Databricks Analytics Handover

Project Summary

This project implements an end-to-end Databricks lakehouse workflow for NYC green and yellow taxi trip analytics. The workflow ingests raw trip files, standardizes service-specific schemas, enriches trips with zone and borough attributes, produces business insights, and trains machine learning models from engineered trip features.

Technical Stack

- ? Databricks for notebook execution and managed lakehouse workflow.
- ? PySpark for distributed transformation and analytics.
- ? Delta Lake for curated table persistence.
- ? Unity Catalog volumes for managed file storage.
- ? Spark SQL for stakeholder-facing analytical queries.
- ? Spark ML and custom feature preparation for predictive modeling.

Data Workflow

1. Configure Spark for NYC-local time semantics.
2. Define reusable catalog, schema, and volume paths.
3. Ingest green and yellow taxi Parquet data.
4. Load taxi zone lookup reference data.
5. Harmonize schemas and union taxi color datasets.
6. Derive duration, distance, speed, temporal, and fare-efficiency features.
7. Apply data quality rules with row-retention visibility.
8. Persist a borough-enriched curated table for analytics and modeling.

Analytics Coverage

- ? Monthly trip and revenue trends.
- ? Descriptive statistics by taxi color.
- ? Route demand grids by borough pair, weekday, month, and hour.
- ? Top pickup-to-dropoff revenue pairs.
- ? Tip behavior and high-tip trip share.

? Speed and distance-efficiency analysis by duration band.

Machine Learning Coverage

? Time-aware train, validation, and test splits.

? Route-time hierarchical baseline.

? Train-only target encoding to avoid leakage.

? Feature standardization and vector assembly.

? Model training, validation, and test evaluation using RMSE.

Operational Notes

? Update notebook configuration if Unity Catalog names differ from the documented defaults.

? Re-run the notebook from top to bottom after changing ingestion paths or feature logic.

? Regenerate the HTML report after material notebook updates.

? Move stable logic into ``src/`` modules before scheduling the workflow as a Databricks job.

Recommended Next Steps

? Add automated data quality checks for schema drift and invalid trip values.

? Register the model workflow with MLflow.

? Package reusable transformations under ``src/transformation`` and ``src/feature_engineering``.

? Convert the notebook into a scheduled Databricks Workflow when the pipeline is ready for recurring execution.